

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

INFORME INDIVIDUAL DE LAS EXPOSICIONES: HERRAMIENTAS CASE PARA GENERACIÓN DE CÓDIGO (MDD)

MATERIA:

INGENIERÍA EN REQUERIMIENTOS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN, PhD.

EQUIPO G

ESTUDIANTE:

FUERTES ARRAES EDSON DANIEL, CI: 0929115087, Correo: efuertes2@uteq.edu.ec

CURSO:

CUARTO SEMESTRE INGENIERÍA EN SOFTWARE “B”

Repositorio GitHub:

https://github.com/MiloSaurio4kHD/Informe_Exposiciones_Herramientas_Case_Fuertes_Grupo_G

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1.	Introducción	3
2.	Grupo A – Umbrello	3
2.1.	Descripción de la herramienta	3
2.2.	Demostración	3
3.	Grupo B – StarUML	3
3.1.	Marco conceptual expuesto	3
3.2.	Ventajas y limitaciones	3
3.3.	Demostración	4
4.	Grupo E – Modelio	4
4.1.	Descripción de la herramienta	4
4.2.	Demostración	4
4.3.	Observaciones	4
5.	Grupo H – BOUML	4
5.1.	Descripción de la herramienta	4
5.2.	Demostración	5
6.	Grupo D – Power Designer	5
6.1.	Descripción de la herramienta	5
6.2.	Demostración	5
7.	Cuadro comparativo de las herramientas	5
8.	Conclusiones	6

Índice de tablas

1.	Comparación de las herramientas CASE expuestas.	6
----	---	---

1 Introducción

El presente informe recoge, de forma individual, lo observado durante las exposiciones de los distintos equipos sobre herramientas CASE aplicadas al desarrollo dirigido por modelos (MDD), es decir, a la generación de código fuente a partir de diagramas UML. Cada equipo presentó una herramienta diferente y demostró, sobre el proyecto MundiPets, el ciclo de modelar un diagrama de clases y obtener a partir de él el esqueleto del código. El objetivo del documento es dejar constancia de las características técnicas de cada herramienta, del procedimiento demostrado en vivo y de las observaciones más relevantes sobre sus ventajas y limitaciones.

2 Grupo A – Umbrello

Expositores: Esther Muñoz, Frixon Morán y Ángelo Zambrano. **Herramienta CASE:** Umbrello.

2.1 Descripción de la herramienta

Esther Muñoz explicó qué es Umbrello, mencionando que es de código abierto, que soporta varios tipos de diagramas UML y que está disponible para Windows y Linux. Frixon Morán explicó cómo Umbrello convierte un modelo a código y expuso los lenguajes a los que se puede generar. Por su parte, Ángelo Zambrano habló sobre las ventajas que brinda la herramienta, como su carácter de código abierto y el soporte de múltiples lenguajes, y detalló sus fortalezas y limitaciones.

2.2 Demostración

En su demostración, Esther Muñoz creó una clase con sus atributos y métodos, y explicó cómo crear un estereotipo y agregarlo a las clases, además de cómo relacionar las clases y manejar las multiplicidades. Frixon Morán, en la parte práctica, mostró cómo crear una clase con todos sus atributos y métodos.

Ángelo Zambrano abrió otro proyecto con el diagrama completo y explicó cómo generar código a partir del modelo: seleccionó el modelo, modificó las configuraciones del generador, seleccionó las clases a generar y ejecutó la generación. Posteriormente creó un proyecto en Visual Studio y agregó las clases generadas.

3 Grupo B – StarUML

Expositores: Génesis Gutiérrez, Mayra Córdova y Anderson Naranjo. **Sistema:** MundiPets.

3.1 Marco conceptual expuesto

Génesis Gutiérrez introdujo el concepto de herramientas CASE, definiéndolas como software diseñado para automatizar y facilitar la documentación de software. A continuación presentó los conceptos clave, paradigmas y arquitecturas del desarrollo dirigido por modelos, señalando que MDE es el enfoque más amplio, en el cual todos los modelos se centran en el proceso de software; que en MDD los modelos generan el sistema; y que MDA separa el diseño conceptual de la lógica de negocio. Explicó además las transformaciones M2M, entendidas como la conversión automática entre diferentes tipos de modelos, y M2T, como la traducción del diagrama a código.

Anderson Naranjo expuso las técnicas de ingeniería asociadas: *forward engineering*, o generación de código a partir del modelo; *reverse engineering*, o creación de diagramas desde código existente; y *round-trip*, que sincroniza de forma bidireccional, de modo que si se modifica uno se refleja en el otro. Respecto de la herramienta, indicó que la arquitectura de StarUML está basada sobre Node.js y Electron, lo que permite usar extensiones para adaptarse a cualquier lenguaje y la convierte en una aplicación flexible.

3.2 Ventajas y limitaciones

Mayra Córdova detalló las ventajas de StarUML: interfaz intuitiva y ágil, soporte superior en métodos M2T que facilita al usuario el seguimiento del proyecto, extensiones que permiten usar cualquier lenguaje y alineación al estándar UML 2.x.

Entre las limitaciones señaló que se necesita una licencia para las opciones más avanzadas, que son obligatorios ciertos ajustes manuales, ya que requiere participación del usuario para realizar los diagramas o corregir detalles, que presenta un alto consumo de recursos del sistema operativo y que las extensiones pueden llegar a fallar.

3.3 Demostración

Génesis Gutiérrez presentó la aplicación al PFC MundiPets, delimitada al módulo de solicitud de adopción. En la demostración en vivo creó un nuevo proyecto y, en la carpeta *model* a la derecha, mediante clic derecho creó un nuevo diagrama de clases. A la izquierda se encuentran los elementos para diagramar: agregó una clase, le asignó un nombre y le añadió atributos y métodos; luego añadió otra clase que completó de la misma forma. A continuación estableció las relaciones, agregó las cardinalidades y asignó nombres a las relaciones para entender mejor el flujo de trabajo.

Para pasar el diagrama a código, ingresó a *Tools* y luego a *Extension Manager*, donde instaló la extensión de C#; tras ello, en *Tools* aparece la opción C#. Al seleccionarla, eligió la carpeta *model* y la ruta del proyecto de Visual Studio 2022, y agregó las clases como elemento existente dentro de Visual Studio 2022. Finalmente señaló que es necesario realizar ajustes manuales debido a las limitaciones de StarUML.

4 Grupo E – Modelio

Expositores: Nieves Jimmy y Morales Gary. **Sistema:** MundiPets.

4.1 Descripción de la herramienta

Nieves explicó que Modelio es una herramienta para desarrolladores y de código abierto. Morales complementó indicando que trabaja sobre UML, es compatible con UML2 y permite construir diagramas de clase y máquinas de estado, además de ser compatible con BPMN y ArchiMate. Señaló que Modelio depende de una clase para compilar sus códigos y que no es una inteligencia artificial, por lo que no corrige por sí sola los errores de UML; tampoco permite crear diagramas de perfil. Mencionó también que trae por defecto Java Classic y que la generación de código se realiza clase por clase.

4.2 Demostración

En la parte demostrativa, Nieves abrió Modelio y creó un proyecto. Para generar código a través de la clase se accede al apartado de módulo y, al crear un diagrama, se elige entre los distintos tipos disponibles el de clase. Allí aparecen diversos componentes: se arrastra el *class model* al área de trabajo y en ese mismo espacio se añaden atributos y métodos. Al dar clic sobre la clase se pueden modificar propiedades como el nombre de la clase y sus atributos; de la misma forma se agregan las operaciones y las relaciones entre clases.

A continuación, Morales mostró que para generar código se da clic derecho sobre una clase y se selecciona *Java element*, marcando así que la clase puede generar código Java, procedimiento que debe repetirse clase por clase. Con clic derecho sobre la clase aparece la opción *Java designer*, desde donde se realiza la generación. Luego se busca el directorio donde se guardó la clase, en la ruta

C:\usuarios\miusuario\modelio\nombreproyecto\src\, y se abre en el bloc de notas para ver el código. El mismo procedimiento se aplica al diagrama de clases del módulo de MundiPets.

4.3 Observaciones

Los expositores indicaron que el *roundtrip* no funciona de forma correcta porque el programa es muy antiguo, pero que las clases generadas se ejecutan sin problema en IntelliJ IDEA.

5 Grupo H – BOUML

Expositores: Chavarría, Mendoza y Sánchez Cornejo. **Sistema:** MundiPets.

5.1 Descripción de la herramienta

Chavarría explicó por qué el equipo eligió esta herramienta y qué iban a realizar, mencionando que utilizaron C++ para la generación de código. Mendoza describió el subsistema del diagrama UML de MundiPets, que contiene varias clases como Usuario, Propietario, Veterinario, Mascota e Historial Médico, con sus atributos y operaciones, además de la visibilidad y la multiplicidad. Señaló que BOUML permite ingeniería inversa,

generación de código y trabajo multiplataforma, transformando diagramas UML a varios lenguajes como C++, C, Java, Python, PHP y SQL; añadió que tiene una interfaz sencilla y es de código abierto.

Sánchez matizó estas ventajas señalando limitaciones: la interfaz resulta poco intuitiva, la curva de aprendizaje es pronunciada y no posee documentación accesible. La generación de código es parcial, ya que solo genera las partes de la clase pero no las estructuras como tales; el trabajo con frameworks y bases de datos es manual. Advirtió además sobre una regeneración delicada, pues si se modifica y se vuelve a generar el código se elimina lo trabajado en esa clase. Concluyó que es gratuito pero cuenta con una comunidad desactualizada.

5.2 Demostración

Mendoza mostró que para generar un proyecto se abre BOUML y se pulsa *New*, se selecciona la ruta y se guarda, con lo que se crea la carpeta con todos los archivos del proyecto (lenguaje, estereotipos, etc.). Para crear un diagrama se da clic derecho al proyecto en BOUML y se crea el diagrama de clases; luego se selecciona la vista creada y se crea el diagrama, que se abre con doble clic. En la parte superior están todas las opciones: para crear una clase se selecciona *class* y se le asigna un nombre. Para crear un atributo se agrega y se le indica el nombre, y opcionalmente el tipo de dato y la privacidad. Para ver el tipo de dato y el valor que reporta, en *drawing settings*, dentro de *diagram*, se configura que no se oculten los atributos de las clases; de igual forma se puede agregar un estereotipo. Para relacionar clases, el menú superior contiene las relaciones de UML.

Para generar código se da clic derecho a la vista de clases y, en *edit*, se crea un *Source artefact*, procedimiento que se repite clase por clase, tras lo cual el código puede observarse en la carpeta. Finalmente se abre la terminal PowerShell, se navega a la carpeta y se ejecuta el *.exe* generado por BOUML.

Chavarría demostró la edición: para editar un atributo de una clase se da clic derecho sobre el atributo y se edita, o se agrega un nuevo atributo, y se añade un operador o método. Al guardar se observa que las cuatro clases fueron actualizadas y, en la carpeta generada por Mendoza, se ejecuta el comando para ver reflejado lo creado en las clases.

6 Grupo D – Power Designer

Expositor: Contreras Kevin. **Herramienta CASE:** Power Designer.

6.1 Descripción de la herramienta

Contreras Kevin explicó que Power Designer no se centra tanto en el modelado, sino en la conexión de los modelos.

6.2 Demostración

Primero creó el proyecto y explicó los diferentes diagramas que permite modelar Power Designer. Para la demostración seleccionó el diagrama de clases, en el cual creó las clases —como la clase Usuario con todas sus propiedades— y les agregó atributos y operaciones. Luego pasó a un diagrama que ya tenía modelado y lo exportó a código SQL, mostró el código generado en SQL Server y lo ejecutó para que se crearan las tablas.

7 Cuadro comparativo de las herramientas

La Tabla 1 resume las herramientas presentadas por los distintos equipos, según lo observado en cada exposición.

Tabla 1: Comparación de las herramientas CASE expuestas.

Herramienta	Lenguaje demostrado	Fortalezas observadas	Limitaciones observadas
Umbrello	C# (integrado en Visual Studio)	Código abierto; soporta varios tipos de diagramas UML; disponible para Windows y Linux; admite múltiples lenguajes; generador configurable.	Requiere configurar el generador y seleccionar manualmente las clases antes de generar.
StarUML	C# (extensión, hacia Visual Studio 2022)	Interfaz intuitiva y ágil; soporte superior en M2T; extensiones para cualquier lenguaje; alineación a UML 2.x; arquitectura sobre Node.js y Electron.	Licencia necesaria para opciones avanzadas; ajustes manuales obligatorios; alto consumo de recursos; las extensiones pueden fallar.
Modelio	Java	Código abierto; compatible con UML2, BPMN y ArchiMate; clases ejecutables en IntelliJ IDEA.	Generación clase por clase; <i>roundtrip</i> no funciona bien por ser un programa antiguo; no crea diagramas de perfil.
BOUML	C++	Código abierto y gratuito; multiplataforma; genera a C++, C, Java, Python, PHP y SQL; permite ingeniería inversa.	Interfaz poco intuitiva; curva de aprendizaje pronunciada; generación parcial; regeneración delicada (borra lo trabajado); comunidad desactualizada.
Power Designer	SQL	Orientada a la conexión de modelos; exporta a SQL y ejecuta la creación de tablas en SQL Server.	Menor énfasis en el modelado según lo expuesto.

8 Conclusiones

Las exposiciones evidenciaron que, si bien todas las herramientas comparten el propósito de generar código a partir de modelos UML, difieren notablemente en cobertura de lenguajes, facilidad de uso y calidad del ciclo de sincronización modelo–código. Modelio y BOUML coinciden en ser opciones de código abierto orientadas a UML, aunque ambas realizan la generación clase por clase y presentan limitaciones en el *roundtrip* y la regeneración. Umbrello, también de código abierto y multiplataforma, mostró un flujo de generación configurable cuya salida se integró directamente en un proyecto de Visual Studio. StarUML destacó por su interfaz y su modelo de extensiones sobre Node.js y Electron, que le permite generar hacia prácticamente cualquier lenguaje, aunque a costa de licencia para funciones avanzadas, mayor consumo de recursos y ajustes manuales posteriores. Power Designer se distingue por su enfoque hacia la conexión de modelos y la generación directa de esquemas SQL ejecutables.

Un punto transversal a casi todas las demostraciones fue que ninguna herramienta entrega código listo para producción: todas generan el esqueleto estructural —atributos, firmas de métodos y relaciones— y dejan la lógica y los ajustes finales al desarrollador. Asimismo, el grado de soporte del ciclo *round-trip* resultó ser el criterio que más diferencia a unas de otras. En conjunto, las demostraciones permitieron comparar distintos enfoques de MDD sobre un mismo caso —MundiPets— y reconocer que la elección de la herramienta depende del lenguaje objetivo, del alcance del proyecto y del nivel de personalización requerido en la salida generada.